

Adaptive Routing in Energy-Harvesting WSNs

System Architecture, Mathematical Foundation, and Full-Stack Execution Manual

GitHub Repository: https://github.com/santhoshvellore7119-web/WSN_proj | Branch: master | Author: Santhosh

1. Project Overview & Problem Formulation

In classical battery-powered Wireless Sensor Networks (WSNs), sensor node battery levels decrease monotonically. Consequently, classical routing protocols (Dijkstra, LEACH, static maximin DP) evaluate nodes using static battery snapshots. In Energy-Harvesting WSNs (EH-WSNs), nodes recharge dynamically from ambient sources (solar, RF, thermal). Static algorithms falsely reject intermediate relays that have low energy at round start but would harvest sufficient ambient energy *just-in-time* as packets traverse preceding hops. Conversely, static algorithms over-utilize shaded non-harvesting nodes, causing early bottleneck depletion. This project delivers a unified simulation engine, algorithmic innovations, and a modern full-stack web interface.

Subsystem	Technology Stack	Key Role & Capabilities
Simulation Core	Python 3.10+, NumPy, SciPy, Matplotlib	3D Time-Augmented DP, DSU detour repair, 1st-order radio model, LEACH/EH-LEACH, diurnal solar traces, 52 unit tests.
Backend API	FastAPI, Uvicorn, SQLite, Pydantic, SQLAlchemy	Async REST API (Port 8000), background job execution, simulation parameter persistence, Swagger/OpenAPI documentation.
Frontend UI	React 19, TypeScript, Vite, Tailwind CSS, Recharts	Interactive single-page application (Port 3000), live node topology canvas, round-by-round replay, multi-metric time-series charts.
Containerization	Docker, Docker Compose	Multi-service container orchestration bundling backend and frontend for zero-configuration deployment.

2. Key Algorithmic Contributions & Physical Modeling

- **3D Time-Augmented Dynamic Programming ($dp[v][h][t]$):** Formulates a state recurrence over a time-expanded DAG where $dp[v][h][t] = \max_{u \in nbr(v)} \min(dp[u][h-1][t-\delta], E_{proj}(v, t_{curr} + t))$. Operates in polynomial time $O(|E| \cdot H \cdot T)$ and space $O(|V| \cdot H \cdot T)$ (<50 kB RAM), providing provable 2ϵ -approximation bounds.
- **Disjoint-Set Union (DSU) Live Detour Recovery:** Slices local alternate paths around depleted intermediate relays in $O(|E|\alpha(V) + deg(u)\alpha(V))$, maintaining 0% packet loss and achieving a **3.3–6.1× speedup** over global Time-DP recalculation.
- **First-Order Radio Dissipation & Reception Penalty (E_{rx}):** Models physical transmission dissipation $E_{tx} = k \cdot (E_{elec} + \epsilon_{fs} \cdot d^2)$ (or $\epsilon_{mp} \cdot d^4$) alongside unavoidable electronic reception dissipation $E_{rx} = k \cdot E_{elec}$ on all intermediate forwarders.
- **Diurnal Solar & Shadow Trace Replay:** Evaluates 24-hour diurnal solar irradiance profiles (clear sky, intermittent cloudy, overcast) and canopy occlusion fractions ($p_{shadow} \in [0.0, 1.0]$).

3. Prerequisites & Environment Setup

Required Software: Python 3.10+ (tested on Python 3.12), Node.js 18+ & npm, Git. Optional: Docker & Docker Compose.

Step 1: Clone the Repository
git clone https://github.com/santhoshvellore7119-web/WSN_proj.git
cd WSN_proj

Step 2: Create and Activate Virtual Environment
• Windows (Command Prompt / PowerShell):
python -m venv .venv
.venv\Scripts\activate
• Linux / macOS:
python3 -m venv .venv
source .venv/bin/activate

Step 3: Install Python Dependencies
pip install --upgrade pip
pip install -r requirements.txt

Step 4: Verify Installation with Test Suite Tripwire
pytest -q (Expected output: 52 passed in < 3.5s)

4. Running CLI Simulations & Scientific Experiments in CMD

All simulation scenarios, multi-seed statistical tests, and benchmark sweeps can be executed directly from CMD or PowerShell:

Script / Command	Description & Output Artifacts
<code>python main.py --nodes 50 --rounds 200 --harvesting solar --visualize</code>	Runs single 50-node simulation with interactive Matplotlib topology and energy history plots.
<code>python run_experiments.py</code>	Executes the 5 canonical scenarios (Baseline, Solar Unaware, Solar Time-DP, Shadowed, Stochastic). Outputs comparison charts in results/.
<code>python run_multiseed.py</code>	Runs 30-seed Monte Carlo evaluation (350 rounds/seed). Computes paired t-tests, Wilcoxon W, Cohen's d, and 95% Confidence Intervals.
<code>python run_heterogeneity_sweep.py</code>	Sweeps canopy occlusion $p_{shadow} \in [0.0, 1.0]$. Generates sensitivity curves demonstrating the multi-hop E_{rx} reception threshold.
<code>python run_scalability_benchmark.py</code>	Measures empirical latency scaling from N=50 to N=500 nodes with isolated timer blocks (Dijkstra vs DP vs Time-DP vs DSU Detour).
<code>python run_dsu_benchmark.py</code>	Benchmarks DSU detour repair speedup across failure rates (0%–30%) over 10 seeds, showing 3.3–6.1x speedup over Time-DP recomputation.
<code>python run_real_trace_experiment.py</code>	Replays 24-hour diurnal solar irradiance profiles across Clear Sky, Cloudy, and Overcast conditions.
<code>python run_real_world_case_study.py</code>	Runs the Great Duck Island habitat monitoring case study (N=32 motes with spatial canopy heterogeneity).

5. Step-by-Step Setup & Execution in Visual Studio Code (VS Code)

- Open Workspace:** Launch VS Code in the root project folder: `code .`
- Select Python Interpreter:** Press `Ctrl+Shift+P` (or `Cmd+Shift+P` on macOS), type **Python: Select Interpreter**, and choose the virtual environment interpreter at `.\venv\Scripts\python.exe`.
- Split Terminals Setup:** Open the integrated terminal (`Ctrl+``) and split it into two panes (`Ctrl+\`):
 - Terminal Pane 1 (Backend):** `cd backend && python -m uvicorn main:app --reload --port 8000`
 - Terminal Pane 2 (Frontend):** `cd frontend && npm run dev`
- Interactive Debugging (.vscode/launch.json):** Create a launch configuration to set breakpoints in simulation algorithms or API routes:

```
// .vscode/launch.json example configuration:
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: FastAPI Backend",
      "type": "debugpy",
      "request": "launch",
      "module": "uvicorn",
      "args": ["backend.main:app", "--reload", "--port", "8000"],
      "jinja": true
    },
    {
      "name": "Python: Run Experiments",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/run_experiments.py"
    }
  ]
}
```

6. Full-Stack Web Application Development Workflow

The project includes a production-grade full-stack web application with a responsive React frontend and a FastAPI backend.

Backend API Architecture (FastAPI + SQLite on Port 8000):

- Start Backend:** `cd backend && python -m uvicorn main:app --reload --port 8000`
- Interactive API Documentation (Swagger UI):** Open <http://localhost:8000/docs> to inspect and test all endpoints.
- Alternative Redoc Docs:** Available at <http://localhost:8000/redoc>.
- Database Persistence:** Simulation runs, configurations, and summary statistics are automatically stored in `backend/wsn_simulator.db` (SQLite).

Frontend Dashboard Architecture (React 19 + Vite on Port 3000):

- Install Dependencies:** `cd frontend && npm install`
- Start Dev Server:** `npm run dev`
- Web Dashboard URL:** Open <http://localhost:3000> in any browser.
- Vite API Proxy:** `frontend/server.ts` automatically proxies all `/api/*` HTTP requests to the FastAPI backend at `http://127.0.0.1:8000`.

Key Backend REST API Endpoints:

HTTP Method & Route	Request Body / Params	Function & Response Payload
POST /api/run-simulation	JSON: { num_nodes, rounds, harvesting_profile, enable_time_dp, ... }	Triggers asynchronous simulation. Returns job ID and initial status.
GET /api/simulation/{job_id}	Path param: job_id (UUID)	Polls simulation status, topology coordinates, and round-by-round energy telemetry.
GET /api/scenarios	None	Returns list of pre-configured scenario templates (Baseline, Solar, Shadow, Stochastic).
GET /api/history	Query: limit, offset	Fetches paginated past simulation runs from SQLite database.
GET /api/export/{job_id}/csv	Path param: job_id	Downloads full round-by-round simulation metrics as a CSV spreadsheet.

7. Containerized Deployment via Docker Compose

For a zero-dependency setup without configuring local Python or Node environments, run both services with Docker:

```
# Build and boot full-stack services in background:
docker compose up --build -d

# View live container logs:
docker compose logs -f

# Stop all services:
docker compose down
```

• **Web Dashboard:** <http://localhost:3000> | • **FastAPI Backend Docs:** <http://localhost:8000/docs>

8. Troubleshooting & Common Pitfalls

Issue / Error Symptom	Root Cause	Recommended Solution
PowerShell Script Execution Disabled running scripts is disabled on this system	Windows PowerShell default execution policy blocks activate.ps1.	Run in PowerShell: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass then reactivate .venv\Scripts\activate.
Port 8000 or 3000 in Use [Errno 10048] address already in use	A previous backend or frontend process is still bound to the port.	Find and terminate process: • Windows: netstat -ano findstr :8000 then taskkill /PID <PID> /F • Linux/macOS: lsof -ti:8000 xargs kill -9
ModuleNotFoundError: No module named 'src'	Python script executed from a subfolder without root in PYTHONPATH.	Run all experiment scripts from the repository root directory (e.g., python run_experiments.py).
502 Bad Gateway on Frontend Backend service unavailable	Frontend Vite proxy cannot reach FastAPI on port 8000.	Ensure the FastAPI backend is running in Terminal 1 before launching the React dev server in Terminal 2.

9. Quick Command Reference Cheat-Sheet

Task / Operation	Command Line String
Clone Project	git clone https://github.com/santhoshvellore7119-web/WSN_proj.git
Create Virtual Env (Win)	python -m venv .venv && .venv\Scripts\activate
Install Python Requirements	pip install -r requirements.txt
Run Unit Tests (52 tests)	pytest -q
Start FastAPI Backend	cd backend && python -m uvicorn main:app --reload --port 8000
Start React Frontend	cd frontend && npm install && npm run dev
Run 30-Seed Evaluation	python run_multiseed.py
Run Scalability Benchmark	python run_scalability_benchmark.py
Run Docker Full-Stack	docker compose up --build